Web scraping & HTML tables

Friday, June 12, 2026

Today: Friday June 12

Before class

- **Perusall Assignment**: *DC* §16.3: HTML tables & web scraping

Plan for today

- Why scraping, and the ethics of doing it responsibly
- A 5-minute tour of HTML
- `rvest`: `read_html()`, CSS selectors, `html_text2()`, `html_attr()`
- The easy win: `html_table()`
- Cleaning what you scrape (footnotes, times, dates)
- In-class activity

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* Ch. 24 · *DC* Ch. 16 · rvest reference

Why scrape?

Every easy data format is alike

> *"Every easy data format is alike. Every difficult data format is difficult in its own way."* - with apologies to Leo Tolstoy

So we've worked with easily accessible datasets like `penguins`, `flights`, or CSVs that can be quickly imported with `read_csv()`. But a lot of data is only available on **web pages** that are formatted for human eyes rather than `read_csv()`.

Data scraping is gathering data from sources that aren't already in data-frame format; in our context, this is usually translating from the HTML a browser renders into a tidy data frame.

When to scrape (and when not to)

Before you scrape, **always check for a friendlier door**:

1. **Is there a download link?**
2. **Is there an API?**

- Many sites (Spotify, the Census, GitHub) offer an API that returns clean JSON. Often a CRAN package already wraps it (`tidycensus`, `spotifyr`, `gh`).

3. **Only then, scrape the HTML.**

Scraping is flexible but fragile. If a website administrator redesigns its HTML, your code will no longer work. It's best to reserve web scraping for cases when no easier option is feasible.

Good web-scraping citizens

Ethical web-scraping

The rule of thumb from *R4DS*: you're usually on safe ground if the data is

- **public**: you didn't have to log in or click "I agree",
- **non-personal**: not names, emails, locations of identifiable people, and
- **factual**: facts and data aren't copyrightable (creative text and images are).

R4DS real-world cautionary example: 2016 researchers scraped ~70,000 **OkCupid** profiles. These are public, but deeply personal dating profiles. The researchers then released the profiles with no anonymization. It was widely condemned. *Public* does not mean *fair game* or *ethical*.

Be a polite guest

A scraper can fire off thousands of requests in seconds. To a small server that may look like an attack.

- **Throttle**: pause between requests. Don't hammer a server.
- **Cache**: never download the same page twice.
- **Identify yourself** and respect `robots.txt` and terms of service where you reasonably can.

The `polite` package automates all of this for you, pausing, cauching, and checking `robots.txt` for you.

```
# install.packages("polite")
library(polite)
session <- bow("https://www.example.com") # introduce yourself
result  <- scrape(session) # politely fetch
```

Quick HTML tour

What a web page really is

HTML (HyperText Markup Language) is the language behind every web page. Strip away the styling and it's just nested text:

```
<html>
<head>
  <title>Page title</title>
</head>
<body>
  <h1 id='first'>A heading</h1>
  <p>Some text &amp; <b>some bold text.</b></p>
  <img src='myimg.png' width='100' height='100'>
</body>
</html>
```

The whole page is a **tree** of nested elements. Scraping works precisely *because* most pages with data have a consistent, repeating structure.

Anatomy of an element

- **Tags** come in pairs: `<p>` … `</p>`. The content lives in between.
- **Attributes** are `name='value'` pairs inside the start tag.
- Two attributes matter most for scraping:
  - `id`: a *unique* label for one element (`id='first'`)
  - `class`: a *reusable* label shared by many elements (`class='weight'`)

Designers use `id` and `class` to *style* the page. We hijack them to *find our data*.

Example element

cute cats

- Start Tag (<a … >): This opens the element and tells the browser where it begins.
- Attributes (href='/cats', class='link'): These live inside the start tag and provide extra information or settings (like the destination URL or CSS styling classes).
- Contents (cute cats): This is the actual text, image, or nested element that gets displayed and affected by the tag.
- End Tag (): This closes the element, telling the browser where it stops.

Useful tags to recognize

Tag	Meaning
`<h1>`–`<h6>`	headings
`<p>`	paragraph
`<ul>` / `<ol>` / `<li>`	unordered / ordered list / list item
`<a href='...'>`	link (destination in the `href` attribute)
`<img src='...'>`	image (source in the `src` attribute)
`<b>` / `<i>` / `<span>`	bold / italic / inline container
`<table> <tr> <th> <td>`	table, row, header cell, data cell

You don't need to memorize HTML, but it's useful to know enough to spot where your data is hiding. Google or the MDN docs are great resources to help fill in any gaps.

rvest

The rvest workflow

`rvest` (think "harvest") is the tidyverse scraping package. It's installed with the tidyverse but loaded separately:

```
library(rvest)
```

Steps:

1. **`read_html(url)`**: download the page into R

2. **`html_elements(css)`**: select the pieces you want (a *CSS selector*)
3. **`html_element(css)`**: narrow to one piece per observation
4. **`html_text2()` / `html_attr()` / `html_table()`**: pull out the actual values

Practice

`minimal_html()` is a function that allows us to hand write HTML. We'll use it to learn, then use the same tools on real web pages.

```
html <- minimal_html("
  <h1>This is a heading</h1>
  <p id='first'>This is a paragraph</p>
  <p class='important'>This is an important paragraph</p>
")
html
```

```
{html_document}
<html>
[1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-
8 ...
[2] <body>\n<h1>This is a heading</h1>\n  <p id="first">This is a paragraph</ ...
```

Everything that follows works *identically* whether `html` came from `minimal_html()` or `read_html("https://...")`.

CSS selectors: three you'll use constantly

A **Cascading style sheets (CSS) selector** is a mini pattern language that provides a concise way of describing which HTML elements you want to extract from a web page.

Selector	Selects	Read it as
`p`	all `<p>` elements	"every paragraph"
`.important`	all elements with `class="important"`	"anything classed important" (dot = class)
`#first`	the element with `id="first"`	"the one with id first" (hash = id)
`p.important`	`<p>` elements that are *also* `class="important"`	combine them
`li b`	`<b>` *inside* an `<li>`	descendant (space)

```r
html |> html_elements("p") # both paragraphs
```

```
{xml_nodeset (2)}
[1] <p id="first">This is a paragraph</p>
[2] <p class="important">This is an important paragraph</p>
```

```r
html |> html_elements(".important") # just the important one
```

```
{xml_nodeset (1)}
[1] <p class="important">This is an important paragraph</p>
```

```r
html |> html_element("#first") # the one with id="first"
```

```
{html_node}
<p id="first">
```

element vs. elements (this trips everyone up)

- **`html_elements()`** (plural): returns *all* matches. Length depends on the page.
- **`html_element()`** (singular): returns *one match per input*, filling `NA` when nothing matches.

```r
html |> html_elements("b") # 0 matches -> empty
```

```
{xml_nodeset (0)}
```

```r
html |> html_element("b") # 0 matches -> NA (length preserved!)
```

```
{xml_missing}
<NA>
```

That `NA`-filling behavior keeps columns aligned.

Why the distinction matters

Below is a page of four StarWars characters, some of whom are missing a recorded weight:

```
droids <- minimal_html("
  <ul>
    <li><b>C-3PO</b> is a <i>droid</i> that weighs <span class='weight'>167 kg</span></li>
    <li><b>R2-D2</b> is a <i>droid</i> that weighs <span class='weight'>96 kg</span></li>
    <li><b>Yoda</b> weighs <span class='weight'>66 kg</span></li>
    <li><b>R4-P17</b> is a <i>droid</i></li>
  </ul>
")
```

Grab the weights *directly* and you get **3** values for **4** characters and no way to know which one is missing:

```
droids |> html_elements(".weight") |> html_text2()   # length 3 - misaligned!
```

```
[1] "167 kg" "96 kg"  "66 kg"
```

Rows then columns

Step 1. Use `html_elements()` to grab one element *per observation* (each `<li>` is one character):

```
characters <- droids |> html_elements("li")
length(characters) # 4 (one per character)
```

```
[1] 4
```

Step 2. Use `html_element()` *on each* to pull one value per observation. Missing weights become `NA`, so everything stays aligned:

```
tibble(
  name = characters |> html_element("b") |> html_text2(),
  species = characters |> html_element("i") |> html_text2(),
  weight  = characters |> html_element(".weight") |> html_text2()
)
```

```
# A tibble: 4 x 3
  name   species weight
  <chr>  <chr>   <chr>
1 C-3PO  droid   167 kg
```

```
2 R2-D2  droid   96 kg
3 Yoda   <NA>    66 kg
4 R4-P17 droid   <NA>
```

> **Tip**
>
> `html_elements()` to find your *rows*, then `html_element()` to find each *column*.

Pulling out the values

Selecting elements gets you HTML nodes. Two functions turn nodes into data:

`html_text2()`: the visible text (handles whitespace like a browser; prefer it over `html_text()`):

```
droids |> html_elements("b") |> html_text2()
```

```
[1] "C-3PO"  "R2-D2"  "Yoda"   "R4-P17"
```

`html_attr("name")`: the value of an attribute, e.g. a link's destination:

```
links <- minimal_html("
  <p><a href='https://en.wikipedia.org/wiki/Cat'>cats</a></p>
  <p><a href='https://en.wikipedia.org/wiki/Dog'>dogs</a></p>
")
links |> html_elements("a") |> html_attr("href")
```

```
[1] "https://en.wikipedia.org/wiki/Cat" "https://en.wikipedia.org/wiki/Dog"
```

> **Warning**
>
> `html_attr()` **always returns a string**. `html_attr("width")` gives `"100"`, not `100`. Follow up with `parse_number()` / `as.integer()`.

Tables

html_table()

If your data is already in an HTML `<table>`, you're in luck. `html_table()` reads it straight into a tibble, guessing column types as it goes.

```
tbl <- minimal_html("
  <table>
    <tr><th>x</th>   <th>y</th></tr>
    <tr><td>1.5</td> <td>2.7</td></tr>
    <tr><td>4.9</td> <td>1.3</td></tr>
    <tr><td>7.2</td> <td>8.1</td></tr>
  </table>
")
tbl |> html_element("table") |> html_table()
```

```
# A tibble: 3 x 2
      x     y
  <dbl> <dbl>
1   1.5   2.7
2   4.9   1.3
3   7.2   8.1
```

`x` and `y` came back as `<dbl>` automatically. (Turn that off with `convert = FALSE` when the auto-guess misfires.)

A real page: world record in the mile

The Wikipedia page on the mile run world record progression holds **a dozen** tables. `html_elements("table")` grabs them all into a list:

```
url <- "https://en.wikipedia.org/wiki/Mile_run_world_record_progression"

tables <- url |>
  read_html() |>
  html_elements("table") |>
  html_table()

length(tables)     # 12 tables on the page
record <- tables[[4]]   # the men's IAAF-era progression
```

```
# A tibble: 6 x 4
  Time   Athlete          Nationality   Date
  <chr>  <chr>            <chr>         <chr>
1 4:14.4 John Paul Jones  United States 31 May 1913[6]
2 4:12.6 Norman Taber     United States 16 July 1915[6]
3 4:10.4 Paavo Nurmi      Finland       23 August 1923[6]
4 4:09.2 Jules Ladoumègue France        4 October 1931[6]
5 4:07.6 Jack Lovelock    New Zealand   15 July 1933[6]
6 4:06.8 Glenn Cunningham United States 16 June 1934[6]
```

Double brackets `[[4]]` because `tables` is a **list** of data frames. (Note: Wikipedia changes frequently, so table *4* today may differ tomorrow; always be sure to look before you trust an index.)

Cleaning scraped data

Scraped data is usually dirty

Look closely at what we got:

- `Date` is `"31 May 1913[6]"`. Here, a footnote marker `[6]` is glued on.
- `Time` is `"4:14.4"`, which is a string, not a number we can plot or subtract.

Scraped values are **always character strings**, often with human-formatting baggage. The next step after scraping is almost always cleaning:

- `str_remove()` / `gsub()` + **regular expressions** to strip junk
- `parse_number()` for currency, percentages, stray units
- `lubridate` (`dmy()`, `mdy()`, …) for dates
- `as.duration(ms(...))` for times like `"4:14.4"`

Stripping footnotes with a regex

We want to delete a bracketed footnote like `[6]` at the end of the date. A **regular expression** describes that pattern:

```r
record <- record |>
  mutate(Date = str_remove(Date, "\\[.\\]$"))
record$Date
```

```
[1] "31 May 1913"    "16 July 1915"   "23 August 1923" "4 October 1931"
[5] "15 July 1933"   "16 June 1934"
```

Decoding `"\\[.\\]$"`:

Piece	Means
`\\[` `\\]`	a *literal* `[` and `]` (escaped — brackets are special)
`.`	any single character (here, the footnote digit)
`$`	anchored at the **end** of the string

We'll discuss regular expressions in more detail next week.

Parsing dates and times

Once we have a clean date, we can use the `lubridate` package. These are `"4 October 1931"`-style to **day-month-year** to `dmy()`:

```
record <- record |>
  mutate(
    Date = dmy(Date),
    seconds = as.numeric(as.duration(ms(Time))) # "4:14.4" -> 254.4 sec
  )
record |> select(Time, seconds, Date)
```

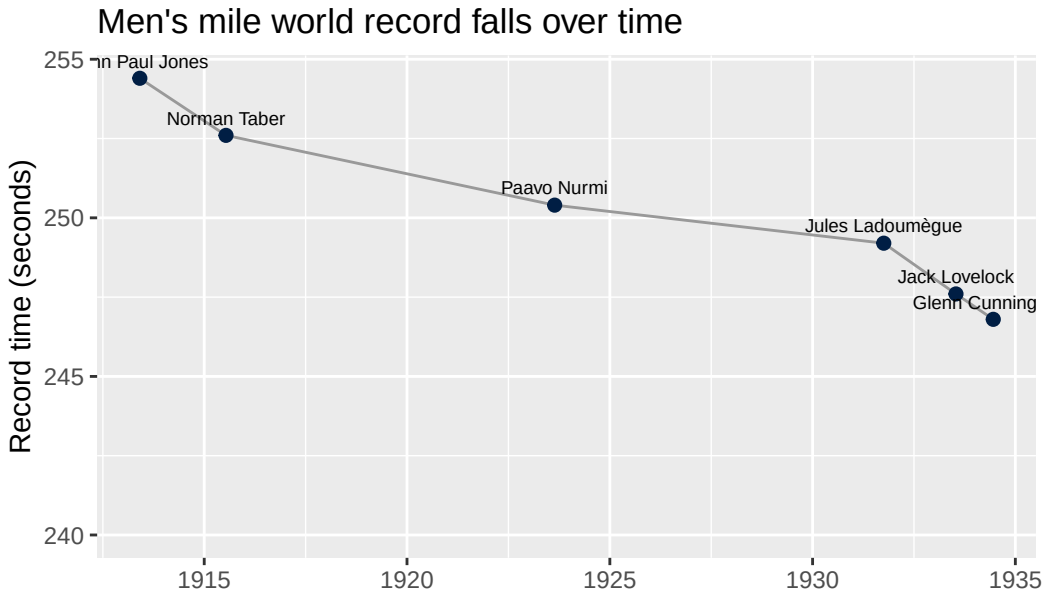
```
# A tibble: 6 x 3
  Time   seconds Date
  <chr>    <dbl> <date>
1 4:14.4    254. 1913-05-31
2 4:12.6    253. 1915-07-16
3 4:10.4    250. 1923-08-23
4 4:09.2    249. 1931-10-04
5 4:07.6    248. 1933-07-15
6 4:06.8    247. 1934-06-16
```

`ms()` reads `"minutes:seconds"`; `as.duration()` turns it into a real time span; `as.numeric()` gives plain seconds we can plot.

Pay-off: a plot of progress

With clean `<date>` and numeric `seconds`, the record progression plots itself:

```
ggplot(record, aes(x = Date, y = seconds)) +
  geom_line(color = "grey60") +
  geom_point(size = 2, color = "#001E44") +
  geom_text(aes(label = Athlete), vjust = -0.8, size = 2.6) +
  labs(
    title = "Men's mile world record falls over time",
    x = NULL, y = "Record time (seconds)"
  ) +
  expand_limits(y = 240)
```



From scattered HTML to a finished plot — the whole import-and-clean pipeline in one slide.

Finding selectors

How do I know *which* selector?

This is the genuinely hard part — and it's normal to need trial and error. Three ways to discover the selector you need:

1. **Right-click then Inspect** in your browser. The developer tools highlight the exact HTML for whatever you clicked. Look at its `class` and `id`.
2. **SelectorGadget**: a point-and-click bookmarklet. Click what you *want* (turns green), click what you *don't* (turns red), and it writes the selector for you.
3. **Copy as selector**: in DevTools, right-click an element → Copy → Copy selector.

You want a selector that's **sensitive** (catches everything you want) *and* **specific** (catches nothing you don't).

Dynamic sites

Sometimes `html_elements()` returns *nothing* like what you see in the browser. Usually that means the page builds its content with **JavaScript** *after* loading.

`rvest` only downloads the raw HTML. It doesn't run JavaScript. This means you should:

- Check whether the data is in the *raw* HTML (View Source, not Inspect).
- If it's truly JS-rendered, you need a tool that drives a real browser: the `chromote` package runs Chrome in the background.
- Or, again: **look for an API first**. Dynamic pages are often fed by one.

Activity

In-class activity

Open `day19-activity.R` in RStudio and work through the parts in order.

- **Part 1**: HTML basics with `minimal_html()`: selectors, `element` vs `elements`, the alignment trap.
- **Part 2**: scrape a real Wikipedia table with `html_table()`, then clean dates and times and make a plot.
- **Part 3 (challenge)**: scrape *non-table* structured content (the rvest StarWars vignette) into a tidy tibble, and pull a link out of an attribute.

Activity file

Wrap-up & looking ahead

What we covered

- Scrape responsibly: public, non-personal, factual; throttle & cache
- HTML is a tree of tags, with `class` and `id` attributes
- `rvest`: `read_html()` → `html_elements()` → `html_element()` → `html_text2()` / `html_attr()`
- The golden rule: **rows with `html_elements()`, columns with `html_element()`**
- `html_table()` for the easy wins
- Clean scraped strings with regex, `parse_number()`, `lubridate`, `as.duration(ms())`

Upcoming

- **Mon Jun 15:** writing your own **functions** (Week 5)
- **HW 4** due Mon Jun 15
- **Project Checkpoint 1** due Sun Jun 14

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 24 · rvest reference · SelectorGadget